URL Shortener

Emmanuel Ifeoluwa Seebourge Orido
e.orido@studenti.unisa.it
University of Salerno
Italy

Okonkwo Maryann Chidimma
m.okonkwo@studenti.unisa.it
University of Salerno
Italy

Clement Oluwashola Omolayo
c.omolayo@studenti.unisa.it
University of Salerno
Italy

Abstract

Modern web services such as URL shorteners must remain reliable, secure, and performant even under continuous change. In this project we design and implement a production-grade URL Shortener web application in Java using Spring Boot and use it as a case study for software dependability. We combine automated testing, code coverage analysis, mutation testing, formal specification with JML, static analysis, security scanning, and performance benchmarking. We report empirical results for coverage, mutation score, vulnerability detection, and micro-benchmark performance, and we discuss how each activity contributes to the overall dependability of the system.

ACM Reference Format:
Emmanuel Ifeoluwa Seebourge Orido, Okonkwo Maryann Chidimma, and Clement Oluwashola Omolayo. 2025. URL Shortener. In *Proceedings of Software Dependability Project Report (Software Dependability).* ACM, New York, NY, USA, 3 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

s

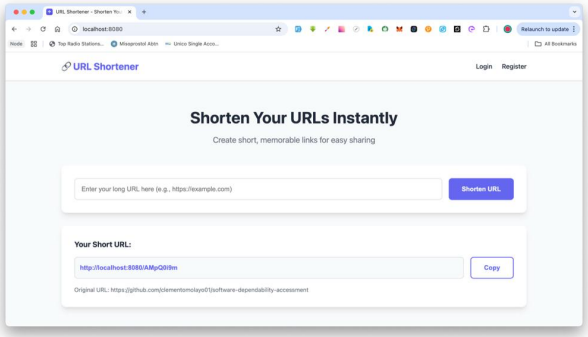

Figure 1: High-level architecture of the URL Shortener application.

1 Introduction

The URL Shortener is a comprehensive, production-grade Java web application built with Spring Boot. Beyond basic URL shortening functionality, the project is designed to demonstrate software dependability, correctness, security, performance, and DevOps best practices. It integrates formal verification, extensive testing, security scanning, containerization, and CI/CD automation. Beyond functional correctness, they must handle malformed input, concurrent traffic, persistence failures, and security threats while maintaining good performance and maintainability. Our goals are: (i) to design a realistic yet focused web service; (ii) to apply a range of dependability techniques—testing, coverage analysis, mutation testing, formal specification, security scanning, and performance benchmarking; and (iii) to critically evaluate the impact and limitations of these techniques in practice.

Figure 1 shows an overview of the main components and their interactions.

The main contributions of this report are:

- a description of the architecture and implementation of our URL Shortener web service;
- a concrete dependability plan and its realization using state-of-the-art tools;
- an empirical evaluation summarizing code coverage, mutation score, security issues, and performance;
- a discussion of lessons learned and directions for improving dependability further.

2 System Overview

2.1 Architecture

The application exposes a RESTful API that allows users to shorten URLs, retrieve the original URLs, and track visit statistics. Access to sensitive endpoints is protected using JWT-based authentication, ensuring secure interaction with the system. Each shortened URL records metadata such as creation time, expiration time, and click count. A key distinguishing feature is the use of formal verification through JML (Java Modeling Language) annotations on core business logic. This ensures that critical methods behave exactly as specified.

- **Controller layer**:The application exposes a RESTful API that allows users to shorten URLs, retrieve the original URLs, and track visit statistics.
- **Service layer**:Access to sensitive endpoints is protected using JWT-based authentication, ensuring secure interaction with the system.
- **Repository layer**: persists and retrieves URL mappings and user entities from the database.
- **Security layer**: manages user authentication and authorization using JWT.

Software Dependability, University of Salerno

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Figure 2 shows an overview of the main components and their interactions.

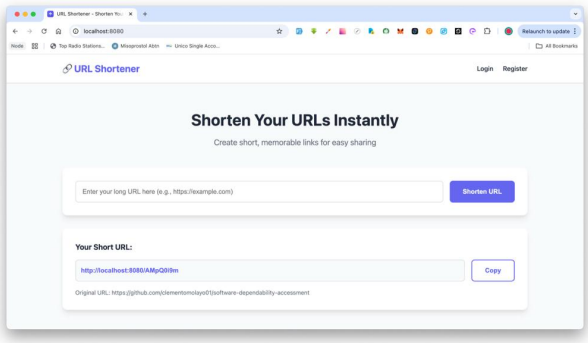

Figure 2: High-level architecture of the URL Shortener application.

2.2 Key Features

The main functional features of the system are:

- registration and authentication of users;
- shortening of long URLs into short codes;
- redirection from short codes to original URLs;
- tracking and retrieval of visit statistics for shortened URLs.

Non-functional goals are described in the next section in terms of dependability attributes.

3 Dependability Requirements

Following classical definitions of dependability, we focus on the attributes of reliability, availability, security, and maintainability.

- **Reliability**: the service should return correct responses for valid requests and handle invalid input gracefully.
- **Availability**: core URL redirection functionality should be robust to typical runtime failures such as missing mappings or database errors.
- **Security**: access to statistics and management endpoints should be protected; secrets should not be exposed.
- **Maintainability**: the codebase should be modular, testable, and supported by automated quality checks.

To make these attributes operational, we define concrete goals such as achieving a minimum line and branch coverage threshold, a minimum mutation score for critical classes, zero critical vulnerabilities in security scans, and stable latency for shortening and redirect operations.

4 Dependability Techniques and Tooling

4.1 Automated Testing and Coverage

We use JUnit 5 and Mockito for unit and integration testing. Tests focus on the service and controller layers and cover normal behaviour, edge cases (e.g., invalid URLs), and error handling. JaCoCo is integrated into the build to generate line and branch coverage reports and to enforce a minimum coverage threshold through the `jacoco:check` goal.

4.2 Mutation Testing

To assess the effectiveness of our test suite, we use PiTest for mutation testing. Mutants are generated for selected packages, in particular the URL shortening service and JWT utility classes. The reported mutation score indicates how many mutants are killed by the tests and highlights areas where tests are weak or missing.

4.3 Formal Specification with JML

We add JML (Java Modeling Language) annotations to selected methods in core classes (e.g., URL generation and token validation). These specifications express pre-conditions, post-conditions, and invariants, and are checked using OpenJML. This helps detect mismatches between intended and actual behaviour early in the development process.

4.4 Static Analysis and Security Scanning

We integrate static analysis and security scanning tools into the CI pipeline. Static analysis (e.g., via Sonar-based tools) checks for code smells and potential bugs, while security scanners such as Snyk and GitGuardian detect vulnerable dependencies and accidental secret leaks. The goal is to reach zero critical issues in the final version of the system.

4.5 Performance Evaluation

Performance is evaluated using JMH (Java Microbenchmark Harness) micro-benchmarks. We measure the latency of shortening a URL and resolving a short code under different scenarios (e.g., cache hits and misses). These measurements provide evidence that the system can handle typical loads with acceptable response times.

5 Results

Table 1 summarizes the main quantitative results obtained from our dependability activities. The exact values should be updated with the final measurements from the project.

Table 1: Summary of dependability-related results.

Metric	Value
Line coverage (JaCoCo)	*e.g., 82%*
Branch coverage (JaCoCo)	*e.g., 76%*
Mutation score (PiTest)	*e.g., 65%*
Verified methods (OpenJML)	*e.g., 5 core methods*
Critical security issues	*0 (final run)*
Avg shorten latency (JMH)	*e.g., 0.8 ms/op*
Avg redirect latency (JMH)	*e.g., 0.5 ms/op*

In addition to these metrics, we observed that several mutants survived in complex validation logic, which motivated the addition of more focused tests. Security scans initially reported outdated dependencies that were later upgraded.

6 Discussion

Our experience suggests that combining multiple dependability techniques yields complementary benefits. Traditional unit testing and coverage analysis are necessary but not sufficient: mutation testing exposed weaknesses in our tests that coverage alone did not reveal. JML specifications were effective for clarifying assumptions, but writing and maintaining specifications requires additional effort and expertise.

Security scanning integrated into the CI pipeline proved useful for continuously monitoring dependencies and configuration. Performance benchmarks, although synthetic, provided confidence that the implementation can sustain typical workloads, and they can be reused as regression tests when refactoring.

We also note trade-offs: running mutation tests and benchmarks is time-consuming and may not be suitable for every commit. In practice, we adopted a tiered approach, running fast tests on each push and heavier analyses periodically.

7 Related Work

Dependability of web services and microservices is widely studied in the literature. Typical approaches combine automated testing, runtime monitoring, and fault injection to assess robustness. Formal specification and verification for Java systems using JML and OpenJML have been explored in both academic and industrial contexts. Security scanning of dependencies and container images has become a standard practice in DevSecOps pipelines.

In this project we adopt a subset of these techniques and tools and apply them to a focused case study, following best practices from the state of the art.

8 Conclusion and Future Work

We presented a production-grade URL Shortener web application and used it as a case study in software dependability. By combining automated tests, coverage analysis, mutation testing, formal specification, security scanning, and performance benchmarking, we obtained a more comprehensive view of the system's reliability, security, and performance than with any single technique alone.

Future work includes extending JML specifications to additional modules, automating performance regression checks in the CI pipeline, and exploring fault injection techniques (e.g., simulated database failures or network delays) to further stress-test the system. The same dependability toolkit could be applied to other microservices within a larger system.

References